



UNIVERSIDADE FEDERAL RURAL DO SEMI-ÁRIDO
Disciplina: ALGORITMO E ESTRUTURA DE DADOS I

Semestre: 2026.2

Data: ____/____/____
PROFESSORA: Rosana Rego

— Exercício 5: Unidade I —

Vetores de ponteiros para struct

Nome: _____

Nota: _____ /1,0

Objetivo

Este documento descreve o funcionamento do programa `atividade5.c`, atividade final que reúne os conceitos de todas as anteriores em um pequeno sistema de entidades (jogador, inimigos e itens), demonstrando, de forma visual e interativa (usando a biblioteca **raylib**):

- Um **vetor de ponteiros para struct** (`Entidade *vetorEntidades[MAX_ENTIDADES]`), em contraste com o vetor de struct contíguo da atividade 4;
- **Alocação dinâmica individual** de cada *struct* com `malloc` (uma chamada por entidade, não uma única chamada para todas);
- Um **enum** (`TipoEntidade`) para diferenciar jogador, inimigo e item;
- Uma **union** (`ExtraEntidade`) para guardar, no mesmo espaço, o dano de um inimigo **ou** o valor de um item;
- Remoção de um elemento apenas **trocando ponteiros**, sem mover nenhuma *struct* na memória.

1 Vetor de struct *vs.* vetor de ponteiros para struct

Na atividade 4, o vetor de inimigos era um único bloco contíguo (`Inimigo *inimigos = malloc(n * sizeof(Inimigo))`): todas as *structs* ficavam lado a lado na memória. Aqui, cada posição do vetor guarda apenas um **endereço**; a *struct* em si é alocada separadamente:

```
1 // vetor de PONTEIROS para struct: cada posicao aponta para um bloco
2 // alocado individualmente com malloc (nao e um bloco contiguo unico)
3 Entidade *vetorEntidades[MAX_ENTIDADES];
4 int totalEntidades = 0;
```

Essa diferença é a essência da atividade: em um vetor de struct, remover um elemento do meio exige mover (copiar) as *structs* seguintes; em um vetor de ponteiros, basta trocar os endereços guardados nas posições do vetor.

2 Enum + union: a struct Entidade

Jogador, inimigo e item são representados pela mesma *struct*, diferenciados pelo campo **enum** tipo. Um inimigo usa o campo **dano** da **union**; um item usa o campo **valor** – nunca os dois ao mesmo tempo:

```

1 typedef enum {
2     ENTIDADE_JOGADOR,
3     ENTIDADE_INIMIGO,
4     ENTIDADE_ITEM
5 } TipoEntidade;
6
7 typedef union {
8     int dano;    // usado quando tipo == ENTIDADE_INIMIGO
9     int valor;   // usado quando tipo == ENTIDADE_ITEM
10 } ExtraEntidade;
11
12 typedef struct {
13     TipoEntidade tipo;
14     Vector2 pos;
15     float raio;
16     int vida;
17     Color cor;
18     ExtraEntidade extra;
19 } Entidade;

```

3 Alocação individual: criarEntidade

Diferentemente das atividades anteriores, aqui **cada entidade** é alocada com sua própria chamada de malloc, e a função devolve o **ponteiro para struct** recém-criado:

```

1 Entidade *criarEntidade(TipoEntidade tipo, Vector2 pos) {
2     Entidade *e = (Entidade *)malloc(sizeof(Entidade));
3     if (e == NULL) return NULL;
4
5     e->tipo = tipo;
6     e->pos = pos;
7     e->raio = (tipo == ENTIDADE_JOGADOR) ? RAIO_JOGADOR
8              : (tipo == ENTIDADE_INIMIGO) ? 15.0f : 8.0f;
9
10    switch (tipo) {
11        case ENTIDADE_JOGADOR:
12            e->vida = 100;
13            e->cor = BLUE;
14            break;
15        case ENTIDADE_INIMIGO:
16            e->vida = 40;
17            e->cor = MAROON;
18            e->extra.dano = GetRandomValue(5, 15);
19            break;
20        case ENTIDADE_ITEM:
21            e->vida = 1;
22            e->cor = GOLD;
23            e->extra.valor = GetRandomValue(5, 20);
24            break;
25    }
26    return e;
27 }

```

4 Removendo uma entidade: troca de ponteiros

A função `removerEntidade` libera a memória da entidade removida e substitui sua posição pelo **último ponteiro** do vetor – uma operação de custo constante, pois apenas endereços são copiados, nunca *structs* inteiras:

```
1 void removerEntidade(int indice) {
2     if (indice < 0 || indice >= totalEntidades) return;
3
4     free(vetorEntidades[indice]);           // libera o bloco
5     vetorEntidades[indice] = vetorEntidades[totalEntidades - 1];
6     totalEntidades--;
7 }
```

Ciclo de vida da memória

Etapa	Vetor de struct (ativ. 4)	Vetor de ponteiros (ativ. 5)
Alocação	um <code>malloc</code> único para todo o vetor	um <code>malloc</code> por entidade (<code>criarEntidade</code>)
Acesso	direto por índice (<code>vetor[i]</code>)	indireto, via ponteiro guardado (<code>vetorEntidades[i]</code>)
Remoção	mover/deslocar <i>structs</i> inteiras	trocar apenas ponteiros (<code>removerEntidade</code>)
Liberação	um único <code>free(vetor)</code>	um <code>free</code> por ponteiro, em laço

Assim como em todas as atividades anteriores, toda memória alocada com `malloc` precisa ser liberada com `free` – aqui isso é feito em um laço que percorre `vetorEntidades` do início ao fim do programa.

5 Exercícios

Exercício 1 — Ordenar por distância trocando ponteiros

Atualmente `vetorEntidades` não segue nenhuma ordem específica.

Tarefa: implemente uma função que reordene `vetorEntidades` para que a entidade mais próxima do jogador fique sempre em `vetorEntidades[1]` (a posição 0 é sempre o jogador).

Dicas:

- A ordenação deve trocar apenas os **ponteiros** guardados no vetor (`Entidade *tmp = vetorEntidades[i]; ...`), nunca o conteúdo das *structs*.
- Calcule a distância usando `jogador->pos` e `vetorEntidades[i]->pos`.
- Chame essa função uma vez por quadro, antes do laço de desenho.

Exercício 2 — Spawner dinâmico de itens

Atualmente todas as entidades são criadas apenas no início do programa.

Tarefa: ao pressionar a tecla N, uma nova entidade do tipo `ENTIDADE_ITEM` deve ser criada em uma posição aleatória e adicionada ao vetor de ponteiros, enquanto houver espaço disponível.

Dicas:

- Use `IsKeyPressed(KEY_N)` para detectar a tecla.
- Reproveite `criarEntidade` e `adicionarEntidade` já existentes.
- Verifique `totalEntidades < MAX_ENTIDADES` antes de criar uma nova entidade, para não estourar o vetor.